

EXPERIMENT - 8

AIM : Implement a hibernate based application to demonstrate relationships among hibernate entities.

entities

Author.java

```
package org.example.entities;

import jakarta.persistence.*;
import java.util.Set;

@Entity
@Table(name = "authors")
public class Author {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int authorId;

    private String name;

    private String email;

    @ManyToMany(cascade = CascadeType.ALL)
    @JoinTable(
        name = "author_book",
        joinColumns = @JoinColumn(name = "author_id"),
        inverseJoinColumns = @JoinColumn(name = "book_id")
    )
```

```

    )
    private Set<Book> books;

    public int getAuthorId() {
        return authorId;
    }

    public void setAuthorId(int authorId) {
        this.authorId = authorId;
    }

    public String getName() {
        return name;
    }

    public void setName(String name) {
        this.name = name;
    }

    public String getEmail() {
        return email;
    }

    public void setEmail(String email) {
        this.email = email;
    }

    public Set<Book> getBooks() {
        return books;
    }

    public void setBooks(Set<Book> books) {
        this.books = books;
    }
}

```

Book.java

```

package org.example.entities;

import jakarta.persistence.*;
import java.util.Set;

@Entity
@Table(name = "books")
public class Book {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private int bookId;

    private String title;

    private double price;

    @ManyToMany(mappedBy = "books", cascade = CascadeType.A
LL)
    private Set<Author> authors;

    public int getBookId() {
        return bookId;
    }

    public void setBookId(int bookId) {
        this.bookId = bookId;
    }

    public String getTitle() {
        return title;
    }

    public void setTitle(String title) {
        this.title = title;
    }

    public double getPrice() {

```

```

        return price;
    }

    public void setPrice(double price) {
        this.price = price;
    }

    public Set<Author> getAuthors() {
        return authors;
    }

    public void setAuthors(Set<Author> authors) {
        this.authors = authors;
    }
}

```

Main.java

```

package org.example;

import org.example.entities.Author;
import org.example.entities.Book;
import org.hibernate.Session;
import org.hibernate.SessionFactory;
import org.hibernate.Transaction;
import org.hibernate.cfg.Configuration;

import java.util.HashSet;
import java.util.Set;

public class Main {

    public static void main(String[] args) {

        SessionFactory factory = new Configuration()
            .configure("hibernate.cfg.xml")
            .addAnnotatedClass(Author.class)

```

```

        .addAnnotatedClass(Book.class)
        .buildSessionFactory();

Session session = factory.openSession();
Transaction tx = session.beginTransaction();

Author a1 = new Author();
a1.setName("Shrey");
a1.setEmail("shrey@gmail.com");

Author a2 = new Author();
a2.setName("Emma");
a2.setEmail("emma@gmail.com");

Book b1 = new Book();
b1.setTitle("Hibernate");
b1.setPrice(500);

Book b2 = new Book();
b2.setTitle("Java");
b2.setPrice(650);

Set<Book> authorBooks = new HashSet<>();
authorBooks.add(b1);
authorBooks.add(b2);

a1.setBooks(authorBooks);
a2.setBooks(authorBooks);

session.persist(a1);
session.persist(a2);

tx.commit();

session.close();
factory.close();
    }
}

```

hibernate.cfg.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<!DOCTYPE hibernate-configuration PUBLIC
    "-//Hibernate/Hibernate Configuration DTD 3.0//EN"
    "http://www.hibernate.org/dtd/hibernate-configuration-3.0.dtd">

<hibernate-configuration>
    <session-factory>

        <property name="connection.driver_class">org.postgresql.Driver</property>
        <property name="connection.url">jdbc:postgresql://localhost:5432/lab</property>
        <property name="connection.username">postgres</property>
        <property name="connection.password">user1</property>
        <property name="dialect">org.hibernate.dialect.PostgreSQLDialect</property>

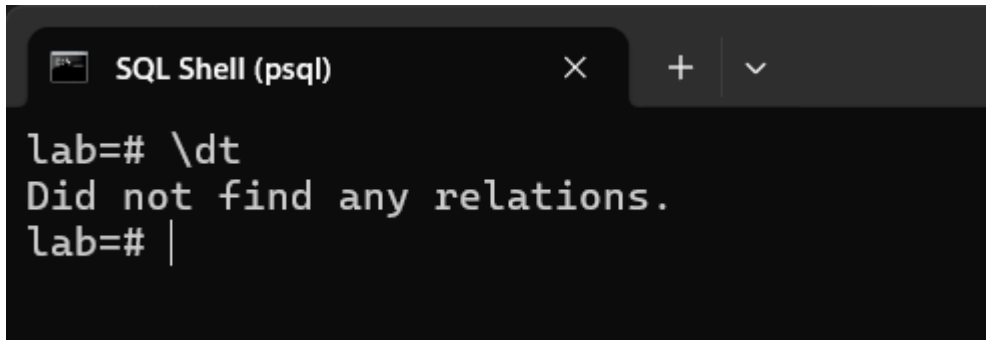
        <property name="show_sql">true</property>
        <property name="format_sql">true</property>
        <property name="hbm2ddl.auto">update</property>

        <mapping class="org.example.entities.Author" />
        <mapping class="org.example.entities.Book" />

    </session-factory>
</hibernate-configuration>
```

OUTPUT

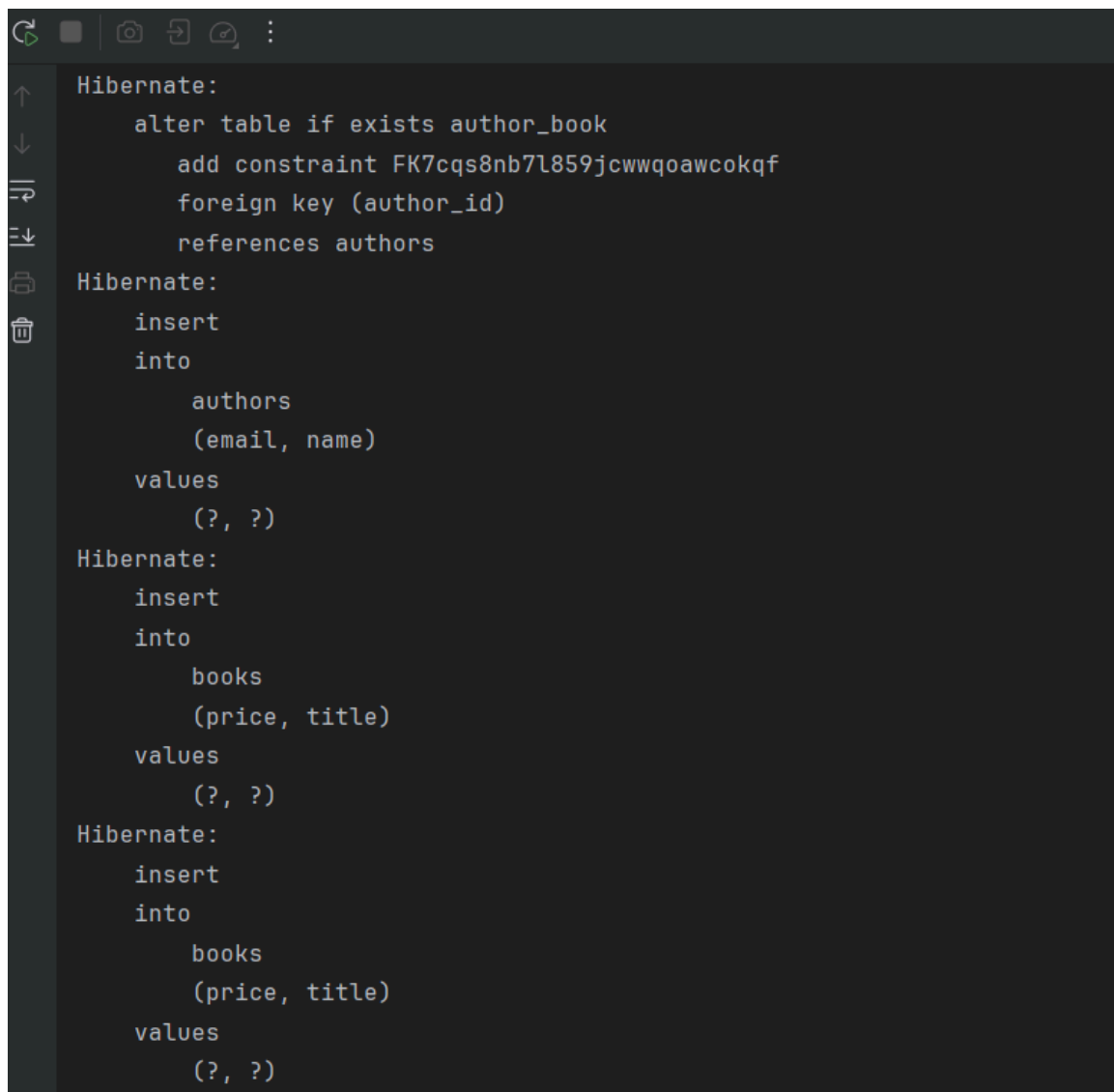
Initially there are no tables in database



```
SQL Shell (psql) × + v
lab=# \dt
Did not find any relations.
lab=# |
```

After running the application, we only save the author, book gets automatically saved

```
Hibernate:
    create table author_book (
        author_id integer not null,
        book_id integer not null,
        primary key (author_id, book_id)
    )
Hibernate:
    create table authors (
        authorId serial not null,
        email varchar(255),
        name varchar(255),
        primary key (authorId)
    )
Hibernate:
    create table books (
        bookId serial not null,
        price float(53) not null,
        title varchar(255),
        primary key (bookId)
    )
Hibernate:
    alter table if exists author_book
        add constraint FKmeehr164a2cpxegeiawuv40a3
        foreign key (book_id)
        references books
```



The image shows a code editor window with a dark theme. On the left side, there is a vertical toolbar with icons for undo, redo, search, and other editing functions. The main area of the editor contains three blocks of SQL code, each preceded by the label "Hibernate:". The first block is an ALTER TABLE statement for "author_book", adding a foreign key constraint "FK7cqs8nb7l859jcwwqoawcokqf" on the "author_id" column, which references the "authors" table. The second block is an INSERT statement into the "authors" table, with columns "email" and "name", and placeholder values "(?, ?)". The third block is an INSERT statement into the "books" table, with columns "price" and "title", and placeholder values "(?, ?)".

```
Hibernate:
    alter table if exists author_book
        add constraint FK7cqs8nb7l859jcwwqoawcokqf
        foreign key (author_id)
        references authors

Hibernate:
    insert
    into
        authors
        (email, name)
    values
        (?, ?)

Hibernate:
    insert
    into
        books
        (price, title)
    values
        (?, ?)

Hibernate:
    insert
    into
        books
        (price, title)
    values
        (?, ?)
```



```
↓
≡
≡↓
🖨️
🗑️
    (?, ?)
Hibernate:
    insert
    into
        authors
        (email, name)
    values
        (?, ?)
Hibernate:
    insert
    into
        author_book
        (author_id, book_id)
    values
        (?, ?)
Hibernate:
    insert
    into
        author_book
        (author_id, book_id)
    values
        (?, ?)
Hibernate:
    insert
    into
        author_book
        (author_id, book_id)
```

```
author_book
(author_id, book_id)
values
(?, ?)
Hibernate:
insert
into
author_book
(author_id, book_id)
values
(?, ?)
Feb 12, 2026 6:46:26 PM org.hibernate.engine.jdbc.connections.internal.DriverManagerConnectionProviderImpl$PoolState stop
INFO: HHH10001008: Cleaning up connection pool [jdbc:postgresql://localhost:5432/lab]

Process finished with exit code 0
```

```
SQL Shell (psql) × + ∨

lab=# select * from authors;
authorid | email | name
-----+-----+-----
1 | shrey@gmail.com | Shrey
2 | emma@gmail.com | Emma
(2 rows)

lab=# select * from books;
bookid | price | title
-----+-----+-----
1 | 650 | Java
2 | 500 | Hibernate
(2 rows)

lab=# select * from author_book;
author_id | book_id
-----+-----
1 | 1
1 | 2
2 | 1
2 | 2
(4 rows)

lab=# |
```